

MODULE

1

Introduction to Power Query M

Understanding the M formula language, the Power Query Editor, and the foundations of data transformation in Power BI and Excel.

Module Contents

■ Module Overview

■ Lesson 1: What Is Power Query M?

- The Power Query Engine
- What Is the M Language?
- Where M Runs: Power BI, Excel, and Dataflows
- M vs. DAX: Choosing the Right Tool
- The M Evaluation Model

■ Lesson 2: The Power Query Editor Interface

- Launching the Power Query Editor
- The Query Pane and Applied Steps
- The Formula Bar
- The Advanced Editor
- Data Preview and Column Profiling

■ Lesson 3: The let...in Expression — Anatomy of a Query

- Structure of a let...in Block
- Step Names and References
- The in Clause and Query Output
- Nesting and Composing Expressions
- Comments in M

■ Lesson 4: Values, Types, and the M Type System

- Everything Is a Value
- Primitive Types
- Structured Types: List, Record, Table
- Type Annotations
- Key Functions: Value.Type, Type.Is, Value.Is

■ Lesson 5: Loading and Shaping Data — Your First Real Query

- Connecting to a Data Source
- Applying Transformations
- Promoting Headers and Changing Types
- Renaming and Removing Columns
- Closing and Loading to the Data Model

■ Demonstration 1: Building Your First M Query from Scratch

■ Demonstration 2: Exploring the Type System

■ Lab 1: Introduction to the M Language

- Exercise 1: Navigate the Power Query Editor
- Exercise 2: Write and Modify a let...in Query
- Exercise 3: Work with Data Types and Primitive Values
- Exercise 4: Shape Data with Basic Transformations

■ Module Review and Takeaways

- Review Questions
- Best Practices
- Common Issues and Solutions
- Key Functions Quick Reference

Module Overview

Power Query M is the formula language that underlies every transformation you perform in the Power Query Editor — whether you are working in Power BI Desktop, Excel, or Azure Data Factory. While the Power Query user interface generates M code automatically as you click through its ribbon commands, a true understanding of M gives you capabilities that the UI alone cannot provide: writing parameterised queries, handling errors gracefully, building reusable custom functions, and diagnosing performance issues at their root.

This module introduces M from the ground up. We begin with the conceptual foundations — what M is, how it evaluates, and where it fits in the Microsoft data stack — before working through the editor interface, query structure, and the type system. By the end of this module you will have written and executed real M queries, understood how every step in the Applied Steps pane maps to an M expression, and loaded your first shaped dataset into the Power BI data model.

NOTE This module assumes you have Power BI Desktop installed (free from microsoft.com/en-us/download). All demonstrations and lab exercises use Power BI Desktop. The same M code runs identically in Excel's Power Query Editor.

Before You Begin

Before starting this module, you should:

- Have Power BI Desktop installed (any recent version — 2023 or later recommended).
- Be comfortable opening files and navigating Windows Explorer.
- Have a basic familiarity with the concept of tabular data (rows and columns).
- Download the lab dataset from the module companion materials (Superstore Sales CSV).

LESSON 1

What Is Power Query M?

After completing this lesson, you will be able to:

- Describe what Power Query M is and where it sits in the Microsoft data stack.
- Explain the difference between M (Power Query) and DAX.
- Describe how M evaluates expressions lazily.
- List the environments in which Power Query M runs.

The Power Query Engine

Power Query is a data connectivity and preparation technology embedded in several Microsoft products. It is designed to let analysts and developers connect to hundreds of data sources, transform raw data into analysis-ready form, and load the results into a target — such as the Power BI data model, an Excel worksheet, or a Dataverse table.

When you click **Get Data** in Power BI Desktop and connect to a CSV file, apply a filter, or rename a column using the ribbon, the Power Query Editor is translating your actions into M formula language code behind the scenes. Every step you see in the Applied Steps pane on the right side of the editor corresponds to exactly one M expression.

The Engine Architecture

Power Query operates as a separate engine from both the storage layer (VertiPaq / DirectQuery) and the DAX engine. Its job is exclusively **Extract, Transform, Load (ETL)** — shaping data before it reaches the model:

- **Extract** — Connect to sources: files, databases, APIs, web pages, cloud services.
- **Transform** — Clean, reshape, combine, and enrich data using M expressions.
- **Load** — Deliver a shaped table to the target (Power BI model, Excel, etc.).

NOTE Power Query runs at **refresh time**, not at report render time. Every time you click Refresh in Power BI Desktop or the Power BI Service refreshes a scheduled dataset, Power Query re-executes all M queries from scratch and delivers fresh data to the model.

What Is the M Language?

M (formally called the **Power Query Formula Language**) is a functional, case-sensitive, whitespace-insensitive expression language designed for data transformation. Its key characteristics are:

Functional	M programs are built from expressions — pieces of code that evaluate to a value. There are no loops, no mutation of variables, and no side effects in the traditional sense.
Case-sensitive	Text, Table, and table are three different identifiers in M. Function names use PascalCase: Table.SelectRows not table.selectrows.
Lazy evaluation	M only evaluates an expression when its value is actually needed. This enables query folding and efficient resource usage.
Composable	Queries can reference other queries, functions can be passed as arguments, and complex transformations are built by composing simple expressions.
Strongly typed	Every value has a type. M checks types at evaluation time and will error if a mismatch occurs — unlike some scripting languages that silently coerce.

Where M Runs

The Power Query M engine is embedded in multiple Microsoft products. The M code you write is identical across all of them — only the loading target differs:

Product	Details
Power BI Desktop	The most common authoring environment. Queries load into the VertiPaq in-memory model or connect via DirectQuery.
Power BI Service	Cloud-based scheduled refresh executes your M queries on Microsoft infrastructure. Same M code as Desktop.
Excel (Get & Transform)	Power Query is available under the Data tab in Excel 2016 and later. Loads to worksheets or the Excel data model.
Azure Data Factory	M expressions are used in Power Query activities within ADF pipelines for cloud-scale ETL.
Dataflows (Power BI & Power Apps)	Shared, reusable Power Query transformations stored in the cloud, used by multiple reports or apps.
Analysis Services (Tabular)	Power Query can be used as an M partition source in SSAS Tabular / Azure Analysis Services models.

M vs. DAX: Choosing the Right Tool

A common point of confusion for new Power BI developers is when to use M (Power Query) versus DAX (Data Analysis Expressions). They serve fundamentally different purposes and operate at different stages of the data lifecycle:

Aspect	M (Power Query)	DAX
Purpose	Data preparation, ETL, cleaning	Calculations, aggregations, time intelligence
When it runs	At refresh time (query execution)	At report render time (query evaluation)
Operates on	Raw source data	Loaded, modelled data
Output	A shaped table loaded to the model	A scalar value, table, or column
Language paradigm	Functional expression language	Formula language (similar to Excel)
Use for	Removing columns, filtering rows, joining tables, changing types, creating date tables	Measures, KPIs, calculated columns, row context, filter context

TIP Rule of thumb: if you can do it in Power Query before the data reaches the model, do it there. Removing unnecessary columns, filtering rows, and cleaning text in M reduces the model size and speeds up DAX calculations. Reserve DAX for metrics that need to respond dynamically to user selections (slicers, filters).

The M Evaluation Model

Understanding how M evaluates expressions is fundamental to writing efficient queries. M uses **lazy evaluation**, also called deferred evaluation: an expression is not evaluated until its value is actually required. This has two important practical consequences:

1. Query Folding

When M recognises that a data source can perform a transformation natively (for example, a SQL database can filter rows), it defers execution to the source and generates the equivalent SQL query. This is called **query folding** or **predicate push-down**. The Power Query engine acts as a translation layer, converting M steps into native source queries wherever possible. This dramatically reduces the amount of data transferred from the source to the M engine.

2. Short-circuit Evaluation

In a **let...in** expression, M only evaluates the steps that are needed to produce the final output. If a step is defined but never referenced (directly or indirectly by the **in** clause), it is never executed. This means you can include diagnostic or verification steps in your query without affecting performance — simply comment them out or avoid referencing them in the **in** clause.

WARNING Lazy evaluation is why you should never assume a step executes just because it is defined. If you want to force immediate evaluation (for example, to cache a table that is referenced multiple times), use `Table.Buffer()` to materialise it in memory. This is covered in detail in Module 7.

LESSON 2

The Power Query Editor Interface

After completing this lesson, you will be able to:

- Launch the Power Query Editor from Power BI Desktop.
- Identify and use the key areas of the Power Query Editor interface.
- Read and edit queries in the Formula Bar.
- Open the Advanced Editor and understand its role.
- Use Column Quality, Column Distribution, and Column Profile tools.

Launching the Power Query Editor

The Power Query Editor is a separate window that opens on top of Power BI Desktop. You can open it in three ways:

1. Click **Transform data** on the Home ribbon in Power BI Desktop.
2. Click **Get Data**, connect to a source, and click **Transform Data** in the Navigator.
3. Right-click any query in the Fields pane and select **Edit query**.

TIP The Power Query Editor does not modify your source data. All transformations are non-destructive — they are applied at refresh time on a copy of the data.

The Editor Interface — Key Areas

The Power Query Editor interface is divided into five main areas. Familiarity with each area is essential for efficient query development:

Queries Pane

Left panel

Lists all queries in the current file. Queries can be organised into groups (folders). Right-click a query to rename, duplicate, reference, or delete it. The active query is highlighted — its content appears in the data preview.

Ribbon

Top toolbar

Contains all transformation commands organised into tabs: Home, Transform, Add Column, View. Every ribbon action generates one M step. The Advanced Editor is under Home tab.

Formula Bar

Below ribbon

Shows the M expression for the currently selected Applied Step. You can edit the expression directly here for simple modifications without opening the Advanced Editor. Click the fx icon to add a new step.

Data Preview

Centre area

Shows the output of the currently selected Applied Step as a table. Column headers show the data type icon. Right-click any column header for transformation options.

Query Settings

Right panel

Contains two sections: Properties (query name and description) and Applied Steps (the sequence of transformation steps). Click any step to navigate to it; drag to reorder.

The Formula Bar

The Formula Bar is one of the most important features for learning M, because it shows you exactly what M expression was generated for every step you create through the UI. As you work through transformations, watch the Formula Bar change.

For example, when you click **Remove Rows** → **Remove Top Rows** and enter 2, the Formula Bar shows:

Auto-generated M expression for Remove Top Rows:

```
= Table.Skip("#Previous Step Name", 2)
```

You can edit this expression directly. Change the 2 to a different number and press Enter — the data preview updates immediately. This is the fastest way to make minor adjustments without opening the Advanced Editor.

TIP The Formula Bar can be toggled on or off from the **View** tab. Ensure it is always visible when learning M.

The Advanced Editor

The Advanced Editor shows the complete M query as a single **let...in** expression. This is where you write, paste, and edit full M code. To open it:

Go to the **Home** tab and click **Advanced Editor** (or press **Ctrl+Shift+M** in some versions).

The Advanced Editor has minimal syntax checking — it will alert you to obvious errors (unclosed brackets, missing commas) but will not catch logical errors. It does provide basic syntax highlighting: keywords in blue, string literals in red, comments in green.

Example: A complete query as shown in the Advanced Editor

```
let
Source = Csv.Document(File.Contents("C:\\Data\\sales.csv"),
```

```
[Delimiter=",", Encoding=65001]),  
#"Promoted Headers" = Table.PromoteHeaders(Source, [PromoteAllScalars=true]),  
#"Changed Type" = Table.TransformColumnTypes(#"Promoted Headers",  
{{"Date", type date}, {"Sales", type number}})  
in  
#"Changed Type"
```

Data Profiling: Column Quality, Distribution, and Profile

Power Query provides three data profiling views, available under the **View** tab. These are invaluable for understanding data quality before writing transformations:

Column Quality

Shows the percentage of values that are Valid, Empty (null), or Error. Shown as a horizontal bar below each column header. A column with 100% valid, 0% empty means no null values.

Column Distribution

Shows a histogram of value frequency. Distinct count and unique count appear below the column name. Useful for identifying duplicate values and outliers.

Column Profile

Shows detailed statistics for the selected column at the bottom of the data preview: min, max, average, standard deviation, value distribution list.

WARNING By default, profiling is based on only the first 1,000 rows. To profile the entire dataset, click the status bar text 'Column profiling based on top 1000 rows' and switch to 'Column profiling based on entire data set'.

LESSON 3

The let...in Expression — Anatomy of a Query

After completing this lesson, you will be able to:

- Explain the structure and purpose of a let...in expression.
- Name steps correctly and reference them from other steps.
- Write the in clause to return any step as the query output.
- Nest let...in expressions for readability and scope isolation.
- Add single-line and multi-line comments to M code.

Structure of a let...in Expression

Every Power Query query is, at its core, a single **let...in** expression. This is the fundamental syntactic unit of M. Understanding it completely is the single most important step in learning the language.

The let...in template:

```
let
  StepName1 = expression1,
  StepName2 = expression2,
  StepName3 = expression_referencing_StepName1_and_StepName2
in
  StepName3
```

The **let** keyword opens a block of named expressions (steps). Each step is separated by a comma. There is no comma after the last step. The **in** keyword closes the block and specifies which step's value to return as the query output. The in clause must evaluate to a single value — typically a table, but it can be any M value.

A Concrete Example

Realistic let...in example:

```
let
  // Step 1: Load the CSV file
  Source = Csv.Document(
    File.Contents("C:\\Data\\Superstore.csv"),
    [Delimiter=";", Encoding=65001, QuoteStyle=QuoteStyle.Csv]
  ),
  // Step 2: Promote row 1 to column headers
  PromotedHeaders = Table.PromoteHeaders(Source, [PromoteAllScalars=true]),
```

```
// Step 3: Set correct data types
ChangedTypes = Table.TransformColumnTypes(PromotedHeaders, {
    {"Order Date", type date},
    {"Sales", type number},
    {"Quantity", Int64.Type}
})
in
// Return ChangedTypes as the query output
ChangedTypes
```

Each step name (Source, PromotedHeaders, ChangedTypes) becomes an identifier you can reference in subsequent steps. The final step, ChangedTypes, is named in the **in** clause and becomes the output of the query — the table you see in the data preview and that gets loaded into the data model.

Step Names and the #"..." Syntax

Step names in M are identifiers. A valid simple identifier starts with a letter or underscore and contains only letters, digits, and underscores. However, Power Query's auto-generated step names often contain spaces and special characters because they are derived from UI action descriptions. These **quoted identifiers** are wrapped in #"..." syntax:

Auto-generated step names use the #"..." quoted identifier syntax:

```
let
Source = ...,
#"Promoted Headers" = Table.PromoteHeaders(Source, ...),
#"Changed Type" = Table.TransformColumnTypes(#"Promoted Headers", ...),
#"Filtered Rows" = Table.SelectRows(#"Changed Type", each [Sales] > 100),
#"Removed Other Cols" = Table.SelectColumns(#"Filtered Rows", {"Order ID",
"Sales"})
in
#"Removed Other Cols"
```

TIP When writing M by hand, use simple step names without spaces (Source, LoadedData, FilteredRows) to avoid the #"..." syntax. This makes your code cleaner and easier to read. The UI generates verbose names because it mirrors the action description — you can always rename steps in Query Settings.

The in Clause

The **in** clause determines what the query returns. It is always a single expression — you reference one of the step names. This is a powerful feature: you can temporarily change the **in** clause to return any intermediate step, which lets you inspect the output of any step during development without deleting later steps.

The in clause can return any step:

```
in
// Return the final step:
ChangedTypes

// OR return an intermediate step for debugging:
// PromotedHeaders

// OR return a scalar for a count check:
// Table.RowCount(ChangedTypes)
```

Nested let...in Expressions

let...in expressions can be nested inside other expressions. This is useful for creating local scope — defining helper variables that are only relevant within a particular step:

Nested let...in for local scope:

```
let
Source = ...,
ProcessedTable = let
// Inner let – local scope
Step1 = Table.PromoteHeaders(Source, [PromoteAllScalars=true]),
Step2 = Table.TransformColumnTypes(Step1, {"Sales", type number})
in
Step2, // Inner in returns Step2 as the value of ProcessedTable

FinalTable = Table.SelectRows(ProcessedTable, each [Sales] > 1000)
in
FinalTable
```

Comments in M

M supports two comment styles:

Comment syntax in M:

```
// Single-line comment – everything after // is ignored

/* Multi-line comment
spans multiple lines
useful for longer explanations */

let
// Load the raw data
Source = Csv.Document(File.Contents("..."), [Delimiter=","]),

/* Promote headers and fix types.
The source has a date column that comes in as text. */
WithHeaders = Table.PromoteHeaders(Source, [PromoteAllScalars=true]),
FixedTypes = Table.TransformColumnTypes(WithHeaders, {"Date", type date})
in
FixedTypes
```

TIP Comments are stripped before evaluation — they have zero performance impact. Use them generously, especially to explain the 'why' behind non-obvious transformations. Future-you (and your colleagues) will be grateful.

LESSON 4

Values, Types, and the M Type System

After completing this lesson, you will be able to:

- State that everything in M evaluates to a value.
- Name and describe all primitive M types.
- Describe the three structured types: list, record, and table.
- Apply type annotations to column transformations.
- Use `Value.Type`, `Type.Is`, and `Value.Is` to inspect types at runtime.

Everything Is a Value

The most fundamental rule of M is that **every expression evaluates to a value**. There are no statements, no void functions, and no side effects — only expressions that produce values. This includes things you might not think of as values:

- The number 42 is a value of type **number**.
- The text "Hello" is a value of type **text**.
- A list {1, 2, 3} is a value of type **list**.
- A function (x) => x + 1 is a value of type **function**.
- A table with 10 rows is a value of type **table**.
- Even a **type** is a value — type number is itself a value.

Primitive Types

Primitive types represent single, atomic values. They are the building blocks of all M data. The table below lists every primitive type with its literal syntax, type keyword, and key notes:

Type	Keyword	Literal Example	Notes
Text	<code>type text</code>	<code>"Hello, world"</code>	Unicode string. Immutable. case-sensitive comparisons by default.
Number	<code>type number</code>	<code>42</code> or <code>3.14</code> or <code>1.5e3</code>	64-bit IEEE 754 double. Covers integers and decimals.
Integer (Int64)	<code>Int64.Type</code>	<code>42</code>	Whole number type used for IDs, counts, years.

Logical	type logical	true or false	Boolean. Only two values. Note: lowercase.
Date	type date	#date(2024, 1, 15)	Calendar date. No time component.
Time	type time	#time(14, 30, 0)	Time of day. No date component.
DateTime	type datetime	#datetime(2024, 1, 15, 14, 30, 0)	Combined date and time.
DateTimeZone	type datetimezone	#datetimezone(2024, 1, 15, 14, 30, 0, 9, 0)	DateTime with timezone offset.
Duration	type duration	#duration(1, 2, 30, 0)	Time interval: days, hours, minutes, seconds.
Binary	type binary	#binary({0x48, 0x65, 0x6C})	Raw byte sequence. Used for file contents.
Null	type nullable ...	null	Absence of a value. Compatible with any nullable type.

NOTE The number type covers both integers and decimals. Use Int64.Type when you know a column contains only whole numbers (IDs, years, counts) — this reduces memory usage in the VertiPaq model. Use type number or Currency.Type for financial figures.

Structured Types: List, Record, and Table

Structured types contain multiple values. They are the types you will work with most frequently when manipulating data.

List

A list is an ordered sequence of values. It is denoted by curly braces { }. Lists are zero-indexed: the first element is at index {0}.

```
// A simple list of numbers
MyNumbers = {1, 2, 3, 4, 5}

// A list of text values
Regions = {"East", "West", "Central", "South"}

// A mixed-type list (valid in M)
Mixed = {1, "hello", true, #date(2024, 1, 1)}

// Access by index (zero-based)
FirstRegion = Regions{0} // = "East"
ThirdRegion = Regions{2} // = "Central"
```

Record

A record is an unordered set of name-value pairs (fields). It is denoted by square brackets []. Records are used extensively to represent single rows of a table or configuration objects.

```
// A record representing a single sale
OneSale = [
  OrderID = "CA-2024-001",
  Customer = "John Smith",
  Sales = 1250.00,
  Date = #date(2024, 3, 15)
]

// Access a field by name
TheDate = OneSale[Date] // = #date(2024, 3, 15)
TheSales = OneSale[Sales] // = 1250.00

// Or use Record.Field
AlsoSales = Record.Field(OneSale, "Sales") // = 1250.00
```

Table

A table is a two-dimensional structure with named, typed columns and ordered rows. Every row in a table is a record. Most M queries produce and consume tables. Tables are created from external sources (CSV, SQL, etc.) or constructed inline using the `#table()` intrinsic:

```
// Inline table creation
SmallTable = #table(
  {"Name", "Score", "Pass"}, // Column names
  {
    {"Alice", 92, true},
    {"Bob", 74, true},
    {"Carol", 58, false}
  }
)

// Access a column as a list
Names = SmallTable[Name] // = {"Alice", "Bob", "Carol"}
Scores = SmallTable[Score] // = {92, 74, 58}
```

Type Annotations

M is a dynamically typed language — you can omit type annotations and M will infer types. However, explicitly stating types improves performance (the engine can skip inference), produces cleaner errors, and makes your intent clear. Type annotations appear in two main places:

1. In `Table.TransformColumnTypes` — setting column types in bulk:

```
Table.TransformColumnTypes(Source, {
  {"Order Date", type date},
  {"Sales", type number},
  {"Quantity", Int64.Type},
  {"Customer", type text},
  {"IsReturned", type logical}
})
```

2. As the optional 4th parameter of `Table.AddColumn`:

```
// Without type annotation:  
Table.AddColumn(Source, "Year", each Date.Year([OrderDate]))  
  
// With type annotation – preferred:  
Table.AddColumn(Source, "Year", each Date.Year([OrderDate]), Int64.Type)
```

Key Type Inspection Functions

Three functions are essential for inspecting types at runtime. They are used in defensive programming, error handling, and dynamic query patterns.

Value.Type

Syntax	Value.Type(value as any) as type	
Parameter	Type	Description
value	any	The value whose type you want to inspect.
Returns	Returns the type of the given value.	
Example	Value.Type("hello")	
Result	type text	
NOTE Use to dynamically determine what type of value you are working with — useful in custom functions that must handle multiple input types.		

Type.Is

Syntax	Type.Is(type1 as type, type2 as type) as logical	
Parameter	Type	Description
type1	type	The type to test.
type2	type	The expected type.
Returns	Returns true if type1 is compatible with type2.	
Example	Type.Is(Value.Type(42), type number)	
Result	true	
NOTE	Type.Is checks compatibility, not equality. Type.Is(Int64.Type, type number) returns true because Int64 is a subtype of number.	

Value.Is

Syntax	Value.Is(value as any, type as type) as logical	
Parameter	Type	Description

value	any	The value to test.
type	type	The expected type.
Returns	Returns true if the value conforms to the specified type.	
Example	Value.Is(#date(2024, 1, 1), type date)	
Result	true	
NOTE Equivalent to Type.Is(Value.Type(value), type). Use to branch logic based on input type.		

LESSON 5

Loading and Shaping Data — Your First Real Query

After completing this lesson, you will be able to:

- Connect to a CSV file using `Csv.Document` and `File.Contents`.
- Promote headers using `Table.PromoteHeaders`.
- Apply correct data types using `Table.TransformColumnTypes`.
- Select, rename, and remove columns.
- Filter rows using `Table.SelectRows`.
- Close and apply a query to load data into the Power BI model.

Connecting to a Data Source

The first step in any M query is the **Source** step — connecting to the raw data. Power Query provides hundreds of connector functions. For CSV files stored locally, the standard pattern is:

Csv.Document

Syntax	<code>Csv.Document(source as any, optional columns as any, ...) as table</code>	
Parameter	Type	Description
<code>source</code>	<i>binary</i>	The binary content of the file. Use <code>File.Contents()</code> or <code>Web.Contents()</code> to obtain this.
<code>columns</code>	<i>number or record</i>	Optional. Number of columns, or a record specifying Delimiter, Encoding, QuoteStyle, etc.
Returns	Returns a table containing the CSV data. Row 1 is treated as data (not headers) until you promote headers.	
Example	<code>Csv.Document(File.Contents("C:\\data.csv"), [Delimiter=",", Encoding=65001])</code>	
Result	(table with first row as column names Column1, Column2...)	
NOTE	Encoding 65001 = UTF-8. Encoding 1252 = Windows Western European. Always specify encoding to avoid garbled characters.	

Table.PromoteHeaders

Syntax	Table.PromoteHeaders(table as table, optional options as nullable record) as table	
Parameter	Type	Description
table	table	The source table whose first row should become column headers.
options	record	Optional. Use [PromoteAllScalars=true] to convert all values to text headers.
Returns	Returns a table with the first row promoted to column headers. All columns are renamed.	
Example	Table.PromoteHeaders(Source, [PromoteAllScalars=true])	
Result	(table with proper column names from row 1)	
NOTE	Always use [PromoteAllScalars=true] — without it, numeric or date values in the header row may cause errors.	

Applying Correct Data Types

After promoting headers, all columns typically have type **any** or type **text**. Assigning correct types is essential: it enables proper sorting, filtering, and calculation in both Power Query and the downstream data model.

Table.TransformColumnTypes

Syntax	Table.TransformColumnTypes(table as table, typeTransformations as list, optional culture as nullable text) as table	
Parameter	Type	Description
table	table	The source table.
typeTransformations	list	A list of {column name, type} pairs.
culture	text	Optional. Culture for parsing, e.g. "en-US". Affects date format interpretation.
Returns	Returns a table with the specified columns cast to the specified types.	
Example	Table.TransformColumnTypes(t, {"Sales", type number}, {"Date", type date})	
Result	(table with Sales as number and Date as date)	
NOTE	If a value cannot be converted to the target type (e.g. converting "ABC" to number), the cell becomes an Error. Handle errors with Table.ReplaceErrorValues.	

Selecting, Renaming, and Removing Columns

After loading and typing data, you almost always need to focus on the relevant columns by removing noise and renaming for clarity.

Table.SelectColumns

Syntax	Table.SelectColumns(table as table, columns as any, optional missingField as nullable number) as table	
Parameter	Type	Description
table	table	The source table.
columns	text or list	A single column name or list of column names to keep.
missingField	number	Optional. MissingField.Error (default), MissingField.Ignore, or MissingField.UseNull.
Returns	Returns a table containing only the specified columns, in the order listed.	
Example	Table.SelectColumns(t, {"Order ID", "Sales", "Date"})	
Result	(table with only Order ID, Sales, Date columns)	
NOTE	Use MissingField.Ignore as the 3rd argument if the source schema might not always include all columns — prevents errors on schema changes.	

Table.RenameColumns

Syntax	Table.RenameColumns(table as table, renames as list, optional missingField as nullable number) as table	
Parameter	Type	Description
table	table	The source table.
renames	list	A list of {old name, new name} pairs.
Returns	Returns a table with the specified columns renamed.	
Example	Table.RenameColumns(t, {"cust_id", "CustomerID"}, {"amt", "SalesAmount"})	
Result	(table with renamed columns)	

Table.RemoveColumns

Syntax	Table.RemoveColumns(table as table, columns as any, optional missingField as nullable number) as table	
Parameter	Type	Description
table	table	The source table.
columns	text or list	Column name or list of column names to remove.

Returns	Returns a table with the specified columns removed.
Example	<code>Table.RemoveColumns(t, {"Unnecessary Column", "Another Column"})</code>
Result	(table without the removed columns)
NOTE Prefer <code>Table.SelectColumns</code> when you know the final list of columns you want — it is safer than <code>RemoveColumns</code> when the source schema may change.	

Filtering Rows

`Table.SelectRows` is the core filtering function in M. It applies a condition to every row and returns only those where the condition is true. The condition is expressed as a function (most commonly using the **each** shorthand):

Table.SelectRows

Syntax	Table.SelectRows(table as table, condition as function) as table	
Parameter	Type	Description
table	table	The source table.
condition	function	A function (row) => logical. Receives each row as a record. Use 'each' shorthand for conciseness.
Returns	Returns a table containing only the rows where condition returns true.	
Example	Table.SelectRows(t, each [Sales] > 1000 and [Region] = "West")	
Result	(table with only rows where Sales > 1000 AND Region = West)	
NOTE	The 'each' keyword is shorthand for a function. 'each [Sales] > 1000' means '(row) => row[Sales] > 1000'. The underscore _ also refers to the current row.	

Closing and Applying Your Query

Once you have built and validated your query in the Power Query Editor, you load it to the data model by clicking **Close & Apply** on the Home tab (or **Close & Apply** → **Apply** to keep the editor open).

What happens when you click Close & Apply:

- Power Query executes all enabled queries from their Source step through the in clause.
- The resulting tables are loaded into the VertiPaq in-memory engine (Import mode) or registered as DirectQuery connections.
- Relationships between tables can then be defined in the Model View.
- The data is now available for DAX measures and report visuals.

TIP

You can right-click any query in the Queries pane and uncheck **Enable load** to prevent it from loading to the data model while still allowing other queries to reference it. Use this for staging queries, helper functions, and configuration records that should not appear as tables in the model.

DEMONSTRATION 1**Building Your First M Query from Scratch**

In this demonstration, the instructor builds a complete M query from scratch using the Advanced Editor. You will observe how the let...in structure, type assignments, and basic transformations combine into a working query.

Scenario: Load the Superstore Sales CSV, clean column names, apply correct types, and filter to only the Technology category.

Demonstration Steps**Step 1: Open Power BI Desktop**

Launch Power BI Desktop. You will land on the Start screen. Click **Get data** → **Text/CSV**.

Step 2: Connect to the CSV

Navigate to the Superstore Sales CSV file. In the preview dialog, click **Transform Data** — not Load. This opens the Power Query Editor.

Step 3: Observe the auto-generated M

Click **Advanced Editor** (Home tab). Observe the M code that was automatically generated. It will look like a basic let...in with Source and PromotedHeaders steps.

Step 4: Rename steps for clarity

In the Advanced Editor, rename #"Promoted Headers" to PromotedHeaders (remove the # and quotes). Press Done.

Step 5: Add a type transformation step

After the PromotedHeaders step, type a comma and add a new step:

```
TypedTable = Table.TransformColumnTypes(PromotedHeaders, {"Order Date", type date}, {"Sales", type number}, {"Quantity", Int64.Type}))
```

Step 6: Add a filter step

Add another step to filter for Technology:

```
TechOnly = Table.SelectRows(TypedTable, each [Category] = "Technology")
```

Step 7: Update the in clause

Change the in clause to reference TechOnly.

Step 8: Click Done and observe

Click Done in the Advanced Editor. The data preview shows only Technology rows. Observe the Applied Steps pane now shows your steps.

Step 9: Close and Apply

Click Close & Apply. The table loads into the Power BI model. Switch to Data View to verify the result.

RESULT The Power BI data model contains a Technology table with correctly typed columns: Order Date (date), Sales (number), Quantity (integer). The Advanced Editor shows a clean let...in expression with readable step names.

DEMONSTRATION 2**Exploring the Type System**

This demonstration explores the M type system interactively — using `Value.Type`, `Type.Is`, and `Value.Is` to inspect types, and observing what happens when type conversions fail.

Scenario: Create a series of blank queries that demonstrate type behaviour, then introduce an intentional type error and handle it with `Table.ReplaceErrorValues`.

Demonstration Steps**Step 1: Create a blank query**

In Power Query Editor: Home → New Source → Blank Query. This creates an empty query with just 'in null'.

Step 2: Type inspection expressions

Open the Advanced Editor and enter the following expressions one at a time, pressing Done after each to observe the result:

Step 3: Observe Value.Type results

Change the expression to: `Value.Type(42)` — observe 'type number' displayed. Try `Value.Type("hello")`, `Value.Type(true)`, `Value.Type(#date(2024,1,1))`.

Step 4: Test Type.Is

Enter: `Type.Is(Value.Type(42), type number)` — should return true. Try `Type.Is(Value.Type("hello"), type number)` — should return false.

Step 5: Simulate a type error

Create a table with a mixed column: `#table({"Value"}, {"100"}, {"200"}, {"ABC"}, {"300"}))`. Then apply `Table.TransformColumnTypes` to cast Value to type number. Observe the error on row 3.

Step 6: Handle with ReplaceErrorValues

Add: `Table.ReplaceErrorValues(step_above, {"Value", null})`. Observe that row 3 now shows null instead of an error.

Complete code for Demonstration 2:

```
// Full example for Demonstration 2 – Step 5 & 6
let
// Create a table with a bad value in row 3
RawData = #table({"Value"}, {"100"}, {"200"}, {"ABC"}, {"300"})),

// This step will produce an Error on row 3
AttemptConvert = Table.TransformColumnTypes(RawData, {"Value", type number}),

// Replace the error with null
ErrorHandled = Table.ReplaceErrorValues(AttemptConvert, {"Value", null})
in
```

```
ErrorHandled // Row 3 now shows null
```

RESULT The ErrorHandled table shows: 100, 200, null, 300. The type error on 'ABC' has been gracefully replaced with null rather than crashing the entire query. The Value column type is number.

LAB 1

Introduction to the M Language

In this lab you will apply the concepts from all five lessons in this module. You will navigate the Power Query Editor, write and modify let...in queries, work with data types, and shape a real dataset. Complete the exercises in order.

Lab Scenario

Contoso Analytics has hired you as a Power BI developer. Your first task is to load the company's Superstore sales data from a CSV file into Power BI Desktop, clean it up, apply correct data types, and deliver a properly shaped table ready for the data model.

NOTE Lab files are available at: <https://github.com/datasets/superstore-sales> or from the instructor-provided USB / LMS download. Save the file as Superstore.csv in C:\LabFiles\ before starting.

Exercise 1: Navigate the Power Query Editor (Estimated time: 10 minutes)

In this exercise you will open the Power Query Editor, identify its key areas, and observe how the Applied Steps pane maps to M code.

1. Open **Power BI Desktop**. On the Start screen, click **Get data**.
2. In the Get Data dialog, select **Text/CSV** and click **Connect**.
3. Navigate to **C:\LabFiles\Superstore.csv** and click **Open**.
4. In the preview dialog, click **Transform Data** to open the Power Query Editor.
5. Locate the five key areas described in Lesson 2: Queries Pane, Ribbon, Formula Bar, Data Preview, and Query Settings. Take a moment to identify each.
6. In the Query Settings pane on the right, locate the **Applied Steps** section. You should see two steps: **Source** and **Promoted Headers**.
7. Click on **Source** in Applied Steps. Observe the Formula Bar — it shows the `Csv.Document` expression.
8. Click on **Promoted Headers**. Observe the Formula Bar changes to show `Table.PromoteHeaders`.
9. On the **View** tab, check **Formula Bar** (if not already checked) and **Column quality**. Observe the quality indicators below each column header.
10. On the **View** tab, click **Column profile**. Click on the Sales column header to see its detailed profile at the bottom of the screen.

RESULT You can identify all five key areas of the Power Query Editor. The Applied Steps pane shows Source and Promoted Headers. The Formula Bar updates as you click each step. Column quality indicators are visible above the data preview.

Exercise 2: Write and Modify a let...in Query (Estimated time: 15 minutes)

In this exercise you will open the Advanced Editor and manually modify the auto-generated let...in query.

1. With the Superstore query selected, click **Home** → **Advanced Editor**. Observe the full let...in expression.
2. Identify: the **let** keyword, each step name and its expression, the separating commas, and the **in** clause.
3. In the Advanced Editor, change the step name **"Promoted Headers"** to **PromotedHeaders** (remove the # and quotes). Update the reference in any subsequent step. Click **Done**.
4. Reopen the Advanced Editor. After the last step (add a comma after it) and before the **in** keyword, add the following new step:

```
RowCount = Table.RowCount(PromotedHeaders)
```

5. Change the **in** clause to: **in RowCount**. Click Done. Observe that the data preview now shows a single number (the row count).
6. Change **in** back to **in PromotedHeaders**. Click Done. The table view returns.
7. Add a comment line above the Source step: **// Load Superstore sales data from CSV**.
8. Add a multi-line comment before PromotedHeaders explaining what the step does.
9. Click **Done**. Verify the editor accepts the changes without error.

RESULT The Advanced Editor shows a clean let...in query with readable step names and comments. Switching the in clause between RowCount and PromotedHeaders demonstrates that any step can be returned as output.

Exercise 3: Work with Data Types and Primitive Values (Estimated time: 15 minutes)

In this exercise you will inspect and correct data types in the Superstore dataset.

1. In the Queries pane, right-click the Superstore query and select **Duplicate**. Rename the duplicate to **TypeExplorer**.
2. With TypeExplorer selected, open the **Advanced Editor**.
3. After the PromotedHeaders step (with a comma), add the following step and click Done:

```
// Check the type of the Sales column's first value
SalesType = Value.Type(PromotedHeaders[Sales]{0})
```

4. Change **in** to **in SalesType**. Observe what type is reported (likely **type text** — the CSV brought everything in as text).

5. Change **in** back to **in PromotedHeaders**. Now add a type transformation step:

```
TypedData = Table.TransformColumnTypes(PromotedHeaders, {
    {"Order Date", type date},
    {"Ship Date", type date},
    {"Sales", type number},
    {"Quantity", Int64.Type},
    {"Discount", type number},
    {"Profit", type number}
})
```

6. Update **in** to reference **TypedData**. Click Done.

7. Observe the column header icons — date columns show a calendar icon, numbers show '1.2', integers show '123'.

8. Add a step after TypedData to verify: **VerifySalesType = Value.Type(TypedData[Sales]{0})**. Change **in** to this step. Confirm it now returns **type number**.

9. Change **in** back to TypedData.

RESULT The TypedData step shows correctly typed columns. Date columns display calendar icons, Sales and Profit display as numbers, Quantity as integer. Value.Type() confirms Sales is now type number.

Exercise 4: Shape Data with Basic Transformations (Estimated time: 20 minutes)

In this exercise you will combine all Lesson 5 functions to produce a clean, analysis-ready table from the Superstore data.

1. Go back to the original Superstore query. Open the Advanced Editor.
2. After your existing PromotedHeaders step, add a type transformation step applying all six types from Exercise 3.
3. Add a column selection step keeping only: Order ID, Order Date, Customer Name, Category, Sub-Category, Region, Sales, Quantity, Profit.

```
SelectedCols = Table.SelectColumns(TypedData, {
    "Order ID", "Order Date", "Customer Name",
    "Category", "Sub-Category", "Region",
    "Sales", "Quantity", "Profit"
})
```

4. Add a rename step to rename **Sub-Category** to **SubCategory** (remove the hyphen):

```
RenamedCols = Table.RenameColumns(SelectedCols, {"Sub-Category",
    "SubCategory"})
```

5. Add a filter step to keep only rows where Profit > 0 (profitable orders only):

```
ProfitableOrders = Table.SelectRows(RenamedCols, each [Profit] > 0)
```

6. Update in to reference **ProfitableOrders**.
7. Click Done. Verify the data preview shows only positive-profit rows.
8. In the Query Settings pane, rename the query from **Superstore** to **FactSales**.
9. Click **Close & Apply** to load the table into the Power BI data model.
10. Switch to **Data View** in Power BI Desktop and verify: correct column types, only profitable rows, SubCategory without hyphen.

RESULT The FactSales table is loaded into the Power BI model with 9 columns, correct types, and only profitable orders. The Applied Steps pane shows a clean, well-named sequence: Source → PromotedHeaders → TypedData → SelectedCols → RenamedCols → ProfitableOrders.

Module Review and Takeaways

Review Questions

Answer these questions to confirm your understanding of Module 1. Answers are provided below each question.

What is the difference between M and DAX in Power BI?

Answer: M (Power Query) runs at refresh time and is used to connect, transform, and clean data before it reaches the model. DAX runs at report render time and is used to calculate measures and aggregations that respond to user filter context.

What are the three parts of a let...in expression?

Answer: The let keyword opening the block, one or more named step expressions separated by commas, and the in clause specifying which step to return as output.

Why does M use lazy evaluation, and what is a practical consequence?

Answer: Lazy evaluation means expressions are only evaluated when their value is needed. A practical consequence is query folding: transformations that can be executed by the source system (e.g. SQL WHERE clauses) are pushed to the source, reducing data transfer.

What function would you use to keep only rows where Sales > 500?

Answer: Table.SelectRows(table, each [Sales] > 500)

What does Table.PromoteHeaders do, and when is it needed?

Answer: It promotes the first row of a table to become column headers. It is needed whenever the source (e.g. CSV, Csv.Document) returns the header row as a data row rather than as column names.

What is the difference between type date and type datetime?

Answer: type date represents a calendar date with no time component. type datetime includes both date and time. Use type date for date-only columns (like Order Date) and type datetime for timestamps.

What happens if you try to cast the text value 'N/A' to type number?

Answer: Power Query produces an Error value in that cell. The column type becomes number but the individual cell shows an error. Use Table.ReplaceErrorValues to replace errors with null or another default.

How do you prevent a query from loading into the data model while still using it as a reference?

Answer: Right-click the query in the Queries pane and uncheck Enable load. The query will still execute if referenced by other queries but will not create a table in the data model.

Best Practices

- Always apply `Table.PromoteHeaders` immediately after `Csv.Document` — before any other transformations.
- Set data types explicitly using `Table.TransformColumnTypes` as the second or third step in every query. Never rely on inferred types in production models.
- Use simple, descriptive step names (without spaces) when writing M by hand. This avoids the `#"..."` quoted identifier syntax.
- Remove unused columns early (`Table.SelectColumns` or `Table.RemoveColumns` in the first 3-4 steps) to reduce the data transferred from source and improve query folding.
- Add comments to explain non-obvious transformations — especially any business logic embedded in filter conditions.
- Set `Enable load = false` on staging queries (helper tables, configuration records, intermediate transformations) that should not appear in the data model.
- Use the `in` clause as a debugging tool — temporarily change it to return any intermediate step to inspect its output without deleting subsequent steps.

Common Issues and Solutions

Headers appear as a data row

Cause: The data has an extra metadata row at the top, or `Table.PromoteHeaders` was not applied.

Solution: Apply `Table.PromoteHeaders` with `[PromoteAllScalars=true]`. If there are metadata rows before the header, use `Table.Skip(Source, n)` first.

Date column shows as text

Cause: `Csv.Document` loads all values as text unless `TransformColumnTypes` is applied.

Solution: Add `Table.TransformColumnTypes` with `{"ColumnName", type date}` to cast the column. Specify a culture if the date format is non-US (e.g. "en-GB" for DD/MM/YYYY).

Error values in numeric column

Cause: Source contains non-numeric values (empty strings, 'N/A', '-') in a column being cast to number.

Solution: Use `Table.ReplaceErrorValues` after the type transformation to replace errors with null or 0.

Query folds but is slow

Cause: The filter is applied after a non-folding step (e.g. `Table.Buffer` or a custom column), preventing folding.

Solution: Move filter and column selection steps before any non-folding operations. Check `View Native Query` to confirm folding is active.

Step name conflict: duplicate names

Cause: Two steps have the same name — M does not allow this.

Solution: Rename one of the conflicting steps. If using auto-generated names, Power Query appends numbers to avoid conflicts.

Key Functions — Module 1 Quick Reference

The following table summarises every function introduced in this module. This reference is designed for quick lookup during lab work and exam preparation.

Function	Category	Syntax (simplified)	Returns
Csv.Document	Source	Csv.Document(binary, options)	Table from CSV binary content
File.Contents	Source	File.Contents(path as text)	Binary content of a local file
Web.Contents	Source	Web.Contents(url as text)	Binary content from a URL
Table.PromoteHeaders	Transform	Table.PromoteHeaders(table, options)	Table with row 1 as headers
Table.TransformColumnTypes	Type	Table.TransformColumnTypes(table, pairs)	Table with new column types
Table.SelectColumns	Column	Table.SelectColumns(table, columns)	Table with only named columns
Table.RemoveColumns	Column	Table.RemoveColumns(table, columns)	Table without named columns
Table.RenameColumns	Column	Table.RenameColumns(table, renames)	Table with renamed columns
Table.SelectRows	Row	Table.SelectRows(table, condition)	Table with only matching rows
Table.Skip	Row	Table.Skip(table, count)	Table with first n rows removed
Table.FirstN	Row	Table.FirstN(table, count)	Table with only first n rows
Table.RowCount	Info	Table.RowCount(table)	Number of rows (integer)
Table.ColumnNames	Info	Table.ColumnNames(table)	List of column name strings
Table.ReplaceErrorValues	Error	Table.ReplaceErrorValues(table, replacements)	Table with errors replaced
Value.Type	Type	Value.Type(value)	The type of the given value
Type.Is	Type	Type.Is(type1, type2)	Logical: is type1 a subtype of type2?
Value.Is	Type	Value.Is(value, type)	Logical: does value conform to type?
Record.Field	Record	Record.Field(record, fieldName)	The value of a named field
Table.Buffer	Perf.	Table.Buffer(table)	Cached copy of table in memory

TIP The full function reference for all M functions covered across all 12 modules is available in the companion document: [PowerQuery_M_Formula_Reference.pdf](#)

Module Summary

- Power Query M is a functional, case-sensitive expression language used for ETL (Extract, Transform, Load) at refresh time in Power BI, Excel, and other Microsoft data products.
- Every M query is a let...in expression composed of named steps. The in clause determines the output.
- M uses lazy evaluation — expressions are evaluated only when needed. This enables query folding (push-down to the source) for performance.
- Use M for data preparation. Use DAX for calculations and metrics that must respond to slicer and filter context.
- The Power Query Editor interface has five key areas: Queries Pane, Ribbon, Formula Bar, Data Preview, and Query Settings/Applied Steps.
- M has primitive types (text, number, logical, date, etc.) and structured types (list, record, table). Always assign types explicitly with `Table.TransformColumnTypes`.
- Core transformation functions include: `Table.PromoteHeaders`, `Table.TransformColumnTypes`, `Table.SelectColumns`, `Table.RenameColumns`, `Table.RemoveColumns`, and `Table.SelectRows`.

Next Module Module 2: Working with Text, Numbers, and Conditional Logic — Covers all text manipulation functions (`Text.Combine`, `Text.Split`, `Text.Replace`, `Text.Contains`), number operations (`Number.Round`, `Number.Mod`, `Number.Power`), and if-then-else logic in M.